Table of contents

Series 6: SAT Solving and Modeling

Methods

SAT Solving – DPLL Algorithm

The DPLL algorithm (Davis–Putnam–Logemann–Loveland) is a recursive search method for determining the satisfiability of a propositional formula in conjunctive normal form (CNF).
It is based on:

- **Unit propagation**: if a clause contains only one literal, assign it to satisfy the clause.
- **Pure literal elimination**: if a literal appears only with one polarity in the set of clauses, assign it the value that satisfies all clauses containing it.
- **Variable splitting**: in the absence of a unit clause or pure literal, choose a variable and recursively explore both branches (variable true, then false).
- **Backtracking**: when a branch leads to a contradiction (empty clause), backtrack and try the opposite assignment.

Transformation to Conjunctive Normal Form (CNF)

To apply DPLL, formulas must be converted to CNF, i.e., a conjunction of clauses, each clause being a disjunction of literals.
Typical steps are:

1. Elimination of implications: $A \Rightarrow B$ becomes $\neg A \vee B$.
2. Application of De Morgan's laws: $\neg(A \wedge B)$ becomes $\neg A \vee \neg B$; $\neg(A \vee B)$ becomes $\neg A \wedge \neg B$.
3. Distribution of $\vee$ over $\wedge$: $A \vee (B \wedge C)$ becomes $(A \vee B) \wedge (A \vee C)$.

Modeling in Propositional Logic

Translation of a problem described in natural language into a propositional formula:

- Identify atomic propositions (e.g., "Alice eats candy").
- Formalize problem constraints using logical connectives ($\wedge$, $\vee$, $\Rightarrow$, $\neg$).
- Combine constraints into a single formula to analyze.

Proof by Refutation (Validity of Reasoning)

To show that reasoning "If premises then conclusion" is valid, prove that the formula premises $\Rightarrow$ conclusion is valid (always true).
An effective method:

1. Negate the entire formula: $\neg$(premises $\Rightarrow$ conclusion) $\equiv$ premises $\wedge \neg$conclusion.
2. Put this negation into CNF.
3. Apply a SAT solver (DPLL): if the negation is **unsatisfiable**, then the original formula is **valid**.

Property of Unsatisfiable Clause Sets

A finite set of clauses (without empty clause) that is unsatisfiable necessarily contains:

- At least **one positive clause** (clause where all literals are positive).
- At least **one negative clause** (clause where all literals are negative).

Proof by contradiction: if all clauses are non-positive (resp. non-negative), we can construct an interpretation that satisfies the set.

Reminder

- **De Morgan's Laws**:
$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$
$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$
- **Implication Elimination**:
$$A \Rightarrow B \equiv \neg A \vee B$$

- **Distributivity of $\vee$ over $\wedge$**:

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

- **Conjunctive Normal Form (CNF)**: conjunction ($\wedge$) of clauses, each clause being a disjunction ($\vee$) of literals.
- **Validity**: a formula is valid if it is true in every interpretation. To prove that ψ is valid, show that $\neg\psi$ is unsatisfiable.
- **DPLL**: complete search algorithm for SAT. It uses unit propagation, pure literal elimination, and splitting.

Illustrative Examples

Example 1: Transformation to CNF

Context:
Consider the formula $\phi = (p_1 \wedge q_1) \vee (p_2 \wedge q_2)$. We want to transform it to CNF to count the number of clauses.

Application:

Apply distributivity of $\vee$ over $\wedge$:

$$\begin{aligned}\phi &= (p_1 \wedge q_1) \vee (p_2 \wedge q_2) \\ &\equiv (p_1 \vee p_2) \wedge (p_1 \vee q_2) \wedge (q_1 \vee p_2) \wedge (q_1 \vee q_2)\end{aligned}$$

We obtain a CNF with **4 clauses**.

In general, for $\phi = (p_1 \wedge q_1) \vee \cdots \vee (p_n \wedge q_n)$, the CNF contains 2^n clauses (each clause is a combination choosing p_i or q_i for each i).

Example 2: Application of DPLL

Context:
We want to know if the following formula is satisfiable:

$$\phi = (\neg p \vee q \vee r) \wedge (s \vee p \vee r) \wedge (\neg q \vee \neg r) \wedge (\neg p \vee s) \wedge (\neg q \vee r) \wedge (q \vee \neg s \vee r)$$

Application:

No obvious unit clause or pure literal. We choose variable q for splitting.

- **Branch $q =$ True**: substitute q with $\top$ (remove clauses containing q and remove $\neg q$ from clauses). We get:

$$\phi[q] = (s \vee p \vee r) \wedge \neg r \wedge (\neg p \vee s) \wedge r$$

The unit clause $\neg r$ forces $r =$ False. Substituting $r =$ False:

$$\phi[q, \neg r] = (s \vee p) \wedge (\neg p \vee s) \wedge \text{False} \quad \text{(because clause } r \text{ becomes false)}$$

We get a contradiction, so this branch fails.

- **Branch $q =$ False**: substitute $\neg q$ with $\top$. We get:

$$\phi[\neg q] = (\neg p \vee r) \wedge (s \vee p \vee r) \wedge (\neg p \vee s) \wedge (\neg s \vee r)$$

We choose $p =$ False ($\neg p$). Then:

$$\phi[\neg q, \neg p] = (s \vee r) \wedge (\neg s \vee r)$$

We choose $s =$ True:

$$\phi[\neg q, \neg p, s] = r$$

Unit clause: $r =$ True.

A solution is therefore: $q =$ False, $p =$ False, $s =$ True, $r =$ True. The formula is **satisfiable**.

Example 3: Modeling and Proof by Refutation

Context:
Candy problem: model the constraints and verify if the reasoning "Therefore all three ate candy" is correct.

Application:

Atoms:

- A: Alice eats candy.
- B: Bob eats candy.
- C: Carole eats candy.

Constraints:

1. $S_1 : A \vee B \vee C$ (at least one ate).
2. $S_2 : A \Rightarrow B$ (if Alice eats, then Bob does too).
3. $S_3 : \neg A \Rightarrow \neg B$ (if Alice does not eat, then Bob doesn't either).

4. $S_4 : (A \wedge B) \Rightarrow C$ (if Alice and Bob eat, then Carole does too).
5. $S_5 : (\neg A \wedge \neg B) \Rightarrow \neg C$ (if neither Alice nor Bob eat, then Carole doesn't either).

Conclusion: $S_6 : A \wedge B \wedge C$.

We want to verify whether $(S_1 \wedge S_2 \wedge S_3 \wedge S_4 \wedge S_5) \Rightarrow S_6$ is valid.

We proceed by refutation: consider the negation

$$\neg\psi = S_1 \wedge S_2 \wedge S_3 \wedge S_4 \wedge S_5 \wedge \neg S_6$$

with $\neg S_6 = \neg A \vee \neg B \vee \neg C$.

Apply DPLL to $\neg\psi$ (after conversion to CNF, not detailed here).
The calculation shows that $\neg\psi$ is **unsatisfiable** (both branches lead to contradiction).
Therefore ψ is valid: the reasoning is correct.

Example 4: Property of Unsatisfiable Sets

Context:
Show that any unsatisfiable set E of clauses (without empty clause) contains at least one positive clause and one negative clause.

Application:

Proof by contradiction.

- Suppose E has no positive clause. Then each clause contains at least one negative literal. If we assign **False** to all variables, each clause contains a literal $\neg x$ that becomes true (since x is false), so all clauses are satisfied. Contradiction with unsatisfiability.
- Suppose E has no negative clause. Then each clause contains at least one positive literal. If we assign **True** to all variables, each clause contains a literal x that becomes true, so all clauses are satisfied. Contradiction.

Therefore E must contain both a positive clause and a negative clause.

Synthesis

This series of exercises illustrates fundamental techniques of propositional logic for verification:

- Transformation to CNF allows applying algorithms like DPLL.
- DPLL is an efficient algorithm for solving the SAT problem.
- Modeling allows translating a concrete problem into logical formulas.
- Proof by refutation allows verifying the validity of reasoning.
- A structural property of unsatisfiable sets (presence of positive and negative clauses) is useful for theoretical proofs.

These methods are essential for formal verification of systems, where one often needs to reason about complex logical constraints.